COMP232 Lab 5

Message Authentication Codes

For this lab session, you will learn to use JCA or Python for message authentication. In particular, you will learn two ways of doing this, both of which have been discussed in the lectures:

1. Computing a hash, and sending the encrypted hash along with the message.
2. Using a *Digital Signature*.

> **References:**
>
> - JCA/JCE Reference Manual
> - JCA Digital Signature Algorithm implementation
> - Python Hash Algorithm Documentation
> - Python RSA Documentation

1. RSA encryption and SHA-1 digests

For Java, download, compile and run the following programs:

- `RandomKeyRSAExample.java` -- RSA encryption/decryption with random keys.
- `MessageDigestExample.java` -- SHA-1 hashing.

For Python, download, compile and run the following programs:

- `RSA.py` (RSA encryption/decryption)
- `HASH.py` (SHA-256 hashing).

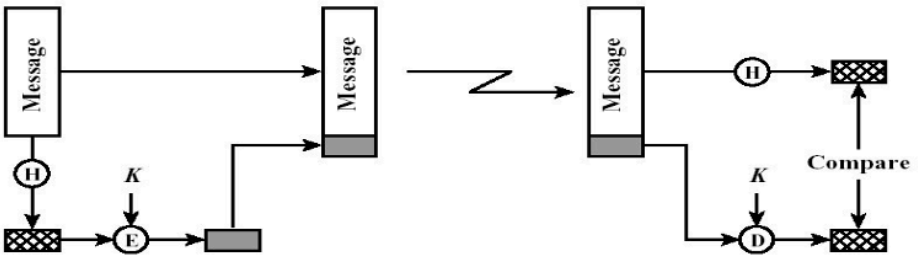
Look into the code and see how the encryption and hashing are implemented.

Write a program which combines both RSA encryption and SHA-1 to demonstrate the message authentication protocol variant (b) shown below (from Lecture 9).

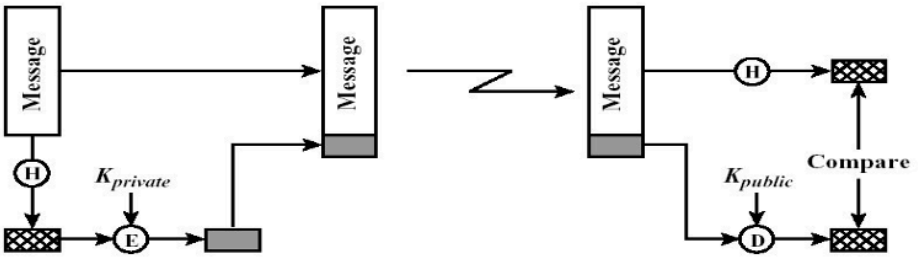
The program needs to model activities of two participants, Sender and Verifier, as follows:

Sender:

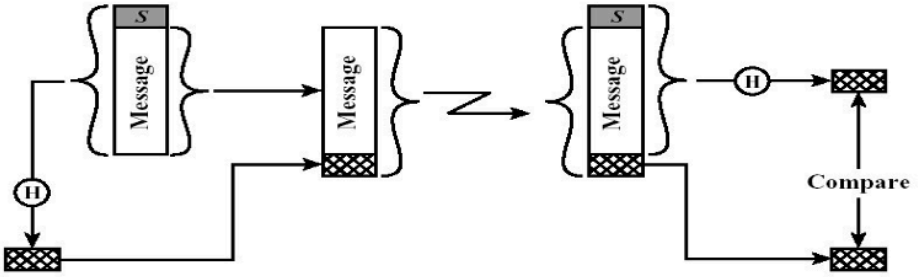
- Takes a text message, and calculates the message digest (hash) using SHA-1
- Generates a *(private key, public key)*-pair using RSA
- Encrypts the message digest with the private key
- Passes the original message, encrypted digest, and public key to the Verifier.



(a) Using conventional encryption



(b) Using public-key encryption



(c) Using secret value

By experimenting with this program, show that the Verifier can detect the following attacks:

- The message has been changed in passing from Sender to Verifier
- The encrypted digest has been changed in passing from Sender to Verifier
- Both message and encrypted digest have been changed.

> **Note:**
>
> In practical scenarios, the public key will indeed be publicly available. It does not have to be sent along with the message.

> **If you are comfortable with programming, you should write two separate functions: `MACDigest`, and `VerifyDigest` as follows:**
>
> 1. `MACDigest` computes the SHA-1 hash of an input message, and then encrypt this hash. It should return the original message, encrypted hash, and public key.
> 2. `VerifyDigest` takes a message, encrypted digest, and public key as input, and verifies the authenticity.

2. Digital Signature Algorithm

Use the Digital Signature Algorithm (DSA) available in JCA or Python to achieve message authentication. See details on use of DSA in:

JCA Digital Signature Algorithm Implementation
Python Digital Signature Algorithm Implementation

3. After the lab (optional)

1. Go ahead and implement the other two variants of MAC discussed in Lecture 9.
2. If you want to practice Object Oriented Programming:
    - Write a class that takes the variant as an argument in the constructor. Then complete this class to obtain a neat implementation of MAC that can return the message digest for any variant on demand.